ENGINEERING_COMPATIBILITY_CHECK

ENGINEERING COMPATIBILITY CHECK — EP-AGENT-001 framework pins

```
ENGINEERING COMPATIBILITY CHECK — NOT AN EP-AGENT-001 EXPERIMENTAL RUN
NO FIXTURE CONSUMED | NO ExecutionProof ENDPOINT CALLED | NO AGENT RUN | NO LLM CALL | NO VERDICT PRODUCED
```

Date: 2026-10-15 | **Host Python:** 3.11.6 (Linux) | **Packet version:** v0.2-DRAFT

1. Boundary: compatibility check vs experimental run

	ENGINEERING COMPATIBILITY CHECK (this document)	**EP-AGENT-001 EXPERIMENTAL RUN**
Purpose	Confirm each pinned package installs and exposes the integration symbols the adapters will bind to	Measure H1–H4 / P1–P5 across the 20 frozen cells
Inputs	Package names and versions only	Frozen fixtures B1–B5, frozen expected results, pinned ExecutionProof
Fixtures B1–B5	**Not loaded, not read, not submitted**	Submitted through each adapter to `/verify`
ExecutionProof endpoint	Not contacted	`/verify`, `/proof/{id}`
Output	Install/import booleans (this file)	Verdicts, ProofRecord refs, metrics, classification
Authorization	Permitted during design	Requires filed lock + Q8/Q9 PASSED + Derek Hone written authorization
Location	Isolated venvs under a scratch directory **outside** the packet; nothing installed into the packet	`results/`, `proofrecords/` (empty; placeholders only)

The structural harness checks (`validate_fixtures.py`, `validate_schemas.py`, `harness_selftest.py`) are also not experimental runs. They read fixture JSON to check structure and normalizer output, call no endpoint, and produce no verdict. This compatibility check does not read fixture files at all.

2. Method

For each framework: create a clean venv → `pip install <package>==<pinned version>` → import the integration-point symbols and introspect the configuration fields that enforce single-agent / single-tool scope. Nothing is instantiated or run.

3. Results (raw; install/import only)

Framework	**Installed**	**Integration symbols importable**	**Scope-control fields present**	**Notes**
OpenAI Agents SDK	`openai-agents==0.24.0` ✔	`agents.Agent` ✔, `agents.function_tool` ✔, `agents.Runner` ✔	`Agent.handoffs` ✔ (left empty), `ModelSettings.parallel_tool_calls` ✔ (set False)	—
LangGraph			n/a (single-node graph by construction)	—

Framework	Installed	Integration symbols importable	Scope-control fields present	Notes
	`langgraph==1.2.15` ✔	`langgraph.prebuilt.ToolNode` ✔, `langgraph.graph.StateGraph` ✔		
CrewAI	`crewai==1.15.28` ✔	`crewai.Agent` ✔, `crewai.Task` ✔, `crewai.tools.BaseTool` ✔	`Agent.allow_delegation` ✔ (set False)	—
Microsoft Agent Framework	`agent-framework==1.22.0` ✔	`FunctionMiddleware` ✔, `FunctionInvocationContext` ✔, `tool` ✔, `FunctionTool` ✔	n/a (one agent, one function tool)	The first probe looked for `agent_framework.ai_fun` it is **not present** in 1.22.0 tool decorator is `agent_framework.tool` / `FunctionTool`. No packet referenced `ai_function`; adapters must bind to `too` `FunctionTool` at implementation (freeze ch item 8)
Google ADK (eligible future extension — NOT INCLUDED IN v1; no change to 20-cell matrix)	`google-adk==2.12.0` ✔ (base package, no `[a2a]` extra)	`google.adk.agents.LlmAgent` ✔, `google.adk.tools.FunctionTool` ✔	`LlmAgent.before_tool_callback` ✔, `disallow_transfer_to_parent` ✔, `disallow_transfer_to_peers` ✔, `sub_agents` ✔ (left empty)	a2a (a2a-sdk) **not installe** with the base package

Interpretation is limited to: these packages install on Python 3.11.6 and expose the listed symbols and fields. No claim is made about integration success, decision fidelity, latency, or any EP-AGENT-001 endpoint.